

10.2

MODIFY [SIMD.EXPOS]

In [simd.expos], insert:

```

template<class From, class To>
    concept simd-consteval-broadcast-arg = see below; // exposition only

template<class V, class T> using make-compatible-simd-t = see below; // exposition only

template<class V>
    concept simd-vec-type = // exposition only
        same_as<V, basic_vec<typename V::value_type, typename V::abi_type>> &&
        is_default_constructible_v<V>;

```

10.3

MODIFY [SIMD.EXPOS.DEFN]

In [simd.expos.defn], insert:

```

template<class From, class To> concept simd-consteval-broadcast-arg = see below;

```

-?- *simd-consteval-broadcast-arg* subsumes *explicitly-convertible-to*.

-?- From satisfies *simd-consteval-broadcast-arg*<To> only if

- *remove_cvref_t*<From> is an arithmetic type,
- From satisfies *convertible_to*<To>,
- the conversion from *remove_cvref_t*<From> to To is not value-preserving, and
- either
 - *common_type_t*<From, To> is To,
 - To is integral and *remove_cvref_t*<From> is int, or
 - To satisfies *unsigned_integral* and *remove_cvref_t*<From> is unsigned int.

10.4

MODIFY [SIMD.OVERVIEW]

In [simd.overview], change:

```

// ([simd.ctor]), basic_vec constructors
template<class explicitly-convertible-to<value_type> U>
    constexpr explicit(see below) basic_vec(U&& value) noexcept;
template<simd-consteval-broadcast-arg<value_type> U>
    consteval basic_vec(U&& x)
template<class U, class UAbi>
    constexpr explicit(see below) basic_vec(const basic_vec<U, UAbi>&) noexcept;

```
